

Pack 20 (Desktop) — First-Connect Device Fingerprint + Viewer Trust Prompt + Trust-on-First-Use (real code)

RemoteDesk · Pack 20 (Desktop) — First-Connect Device Fingerprint + Viewer Trust Prompt + Trust-on-First-Use (real code)

Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 → 19. Everything so far assumes a device already has a `trustStatus`. This slice builds the moment that status is first established: a stable, non-sensitive device fingerprint, a viewer-facing trust prompt on first connect, and a trust-on-first-use (TOFU) decision flow that detects fingerprint CHANGES on a known device and downgrades to untrusted until re-approved. Build order: shared fingerprint/trust contracts → fingerprint builder → trust decision store → trust prompt UI → session wiring → tests.

Scope: establish + verify device identity before trust is granted, and catch identity changes on return visits.

Fail-closed: unknown or changed fingerprint => untrusted (which Pack 15 already uses to deny remote input). No device secrets, IPs, or hardware serials in the fingerprint or logs. Host stays source of truth. No native executor.

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]		
packages/shared/src/permissions/deviceFingerprint.ts		
packages/shared/src/permissions/trustDecision.ts		
apps/desktop/src/renderer/src/services/fingerprintBuilder.ts		
apps/desktop/src/renderer/src/services/trustDecisionStore.ts		
apps/desktop/src/renderer/src/components/TrustPrompt.tsx		
apps/desktop/src/renderer/src/components/FingerprintChangeBanner.tsx		
tests/permissions/deviceFingerprint.test.ts		
tests/permissions/trustDecision.test.ts		
tests/permissions/trustDecisionStore.test.ts		
MODIFIED [MODIFY]		
packages/shared/src/permissions/index.ts	(export deviceFingerprint + trustDecision)	
apps/desktop/src/renderer/src/services/devicePolicyClient.ts	(attach fingerprint on policy fetch)	
apps/desktop/src/renderer/src/components/RemoteSessionView.tsx	(trust prompt + fingerprint-change banner)	
NOT TOUCHED (intentionally)		
apps/desktop/src/main/index.ts	(no native executor; no hardware serial access)	
apps/desktop/src/preload/index.ts	(no new privileged bridge)	
apps/desktop/src/renderer/src/services/webrtc.ts	(Pack 15/17 gates unchanged)	
apps/desktop/src/renderer/src/services/permissionState.ts	(Pack 15/16 unchanged; consumes trustStatus)	
apps/api/** apps/web/**	(trust persistence already exists server-side)	

Part A — Shared: fingerprint + trust contracts

FILE: packages/shared/src/permissions/deviceFingerprint.ts

```
/**
 * Non-sensitive device fingerprint. Built ONLY from coarse, non-identifying
 * signals (platform family, app version, screen bucket, locale, a per-install
 * random id the user can reset). NO IPs, MAC/serials, usernames, or hardware ids.
 * The fingerprint is a stable hash used to recognize a returning install and to
 * detect material changes — not to uniquely track hardware.
 */
export interface FingerprintSignals {
  /** "mac" | "win" | "linux" — family only, not full OS build. */
  platformFamily: string;
  /** App semver, e.g. "1.4.0". */
  appVersion: string;
  /** Coarse screen bucket, e.g. "1920x1080" rounded — not exact geometry. */
  screenBucket: string;
  /** BCP-47 language, e.g. "en-US". */
  locale: string;
  /** Per-install random id (persisted locally, user-resettable). Not hardware-derived. */
  installId: string;
}

export interface DeviceFingerprint {
  /** Stable hash (hex) of the canonical signals. */
  hash: string;
  /** The signals that produced the hash (for change diffing + display). */
  signals: FingerprintSignals;
  /** ISO of when it was computed. */
  computedAt: string;
}

/** Canonical string for hashing: order-stable, lowercased keys. */
export function canonicalizeSignals(s: FingerprintSignals): string {
  return [
    `platform=${s.platformFamily}`,
    `app=${s.appVersion}`,
    `screen=${s.screenBucket}`,
    `locale=${s.locale}`,
    `install=${s.installId}`,
  ].join('&');
}

/**
 * Which signal changes are "material" (should trigger re-trust). installId change
 * is the strongest signal (fresh install / reset). platformFamily change is also
 * material. appVersion/screen/locale drift alone is NOT material on its own.
 */
export function materialSignalChanges(
  prev: FingerprintSignals,
  next: FingerprintSignals,
): string[] {
  const changes: string[] = [];
  if (prev.installId !== next.installId) changes.push('installId');
  if (prev.platformFamily !== next.platformFamily) changes.push('platformFamily');
  return changes;
}
```

```

}

/** True if the change set warrants dropping back to untrusted. */
export function isMaterialChange(prev: FingerprintSignals, next: FingerprintSignals): boolean {
  return materialSignalChanges(prev, next).length > 0;
}

```

FILE: packages/shared/src/permissions/trustDecision.ts

```

import type { DeviceTrustStatus } from './devicePolicy';
import { isMaterialChange, type DeviceFingerprint } from './deviceFingerprint';

/**
 * Trust-on-first-use (TOFU) decision logic. Pure + fail-closed:
 * - first time we see a device => 'prompt' (ask the operator; status untrusted until approved)
 * - known device, same fingerprint => keep persisted status
 * - known device, MATERIAL fingerprint change => 'prompt' + force untrusted
 * - blocked is sticky: a blocked device is never auto-trusted by TOFU
 */
export type TrustGateOutcome = 'trusted' | 'untrusted' | 'blocked' | 'prompt';

export interface TrustEvaluationInput {
  /** Persisted trust status from the API, or null if the device is unknown. */
  persistedStatus: DeviceTrustStatus | null;
  /** Fingerprint stored alongside the persisted status, or null if none. */
  knownFingerprint: DeviceFingerprint | null;
  /** Freshly computed fingerprint for this connection. */
  currentFingerprint: DeviceFingerprint;
}

export interface TrustEvaluation {
  outcome: TrustGateOutcome;
  /** Effective trust status to feed into the policy snapshot (fail-closed). */
  effectiveStatus: DeviceTrustStatus;
  /** Why, for UI + audit (non-sensitive). */
  reason: 'first_use' | 'unchanged' | 'fingerprint_changed' | 'blocked';
}

export function evaluateTrust(input: TrustEvaluationInput): TrustEvaluation {
  const { persistedStatus, knownFingerprint, currentFingerprint } = input;

  // Blocked is sticky and overrides everything.
  if (persistedStatus === 'blocked') {
    return { outcome: 'blocked', effectiveStatus: 'blocked', reason: 'blocked' };
  }

  // Unknown device OR no stored fingerprint => first use => prompt, untrusted now.
  if (persistedStatus === null || knownFingerprint === null) {
    return { outcome: 'prompt', effectiveStatus: 'untrusted', reason: 'first_use' };
  }

  // Known device: detect a material fingerprint change => re-prompt, untrusted.
  if (isMaterialChange(knownFingerprint.signals, currentFingerprint.signals)) {
    return { outcome: 'prompt', effectiveStatus: 'untrusted', reason: 'fingerprint_changed' };
  }
}

```

```

// Stable + known: keep the persisted status (trusted or untrusted).
return {
  outcome: persistedStatus === 'trusted' ? 'trusted' : 'untrusted',
  effectiveStatus: persistedStatus,
  reason: 'unchanged',
};
}

/** The operator's response to a trust prompt. */
export type TrustPromptResponse = 'trust' | 'keep_untrusted' | 'block';

/** Map a prompt response to the resulting persisted status. */
export function statusFromPromptResponse(r: TrustPromptResponse): DeviceTrustStatus {
  switch (r) {
    case 'trust': return 'trusted';
    case 'block': return 'blocked';
    case 'keep_untrusted':
    default: return 'untrusted';
  }
}

```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```

export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval';
export * from './inputAction';
export * from './sessionAuditEvents';
export * from './reconnect';
export * from './fileTransferState';
export * from './clipboardSync';
export * from './deviceFingerprint'; // ADD
export * from './trustDecision';      // ADD

```

Part B — Desktop: fingerprint builder

FILE: apps/desktop/src/renderer/src/services/fingerprintBuilder.ts

```

import {
  canonicalizeSignals,
  type DeviceFingerprint,
  type FingerprintSignals,
} from '@remotedesk/shared';

/**
 * Builds a non-sensitive device fingerprint from coarse renderer-available signals.
 * Deliberately avoids IPs, MAC/serials, usernames, GPU strings, etc. The installId
 * is a per-install random value persisted in localStorage and user-resettable.
 */
const INSTALL_ID_KEY = 'remotedesk.installId';

export async function buildFingerprint(appVersion: string): Promise<DeviceFingerprint> {
  const signals: FingerprintSignals = {
    platformFamily: detectPlatformFamily(),
    appVersion,

```

```

    screenBucket: screenBucket(),
    locale: typeof navigator !== 'undefined' ? navigator.language : 'unknown',
    installId: getOrCreateInstallId(),
  };
  return {
    hash: await sha256Hex(canonicalizeSignals(signals)),
    signals,
    computedAt: new Date().toISOString(),
  };
}

/** Reset the per-install id (e.g. a "forget this device" action). */
export function resetInstallId(): void {
  try {
    localStorage.setItem(INSTALL_ID_KEY, cryptoRandomId());
  } catch {
    /* storage unavailable: next build falls back to an ephemeral id */
  }
}

function detectPlatformFamily(): string {
  const p = typeof navigator !== 'undefined' ? navigator.platform.toLowerCase() : '';
  if (p.includes('mac')) return 'mac';
  if (p.includes('win')) return 'win';
  if (p.includes('linux')) return 'linux';
  return 'unknown';
}

/** Coarse screen bucket: round to the nearest 160px so minor changes don't churn. */
function screenBucket(): string {
  if (typeof screen === 'undefined') return 'unknown';
  const round = (n: number) => Math.round(n / 160) * 160;
  return `${round(screen.width)}x${round(screen.height)}`;
}

function getOrCreateInstallId(): string {
  try {
    const existing = localStorage.getItem(INSTALL_ID_KEY);
    if (existing) return existing;
    const id = cryptoRandomId();
    localStorage.setItem(INSTALL_ID_KEY, id);
    return id;
  } catch {
    return cryptoRandomId(); // ephemeral fallback
  }
}

function cryptoRandomId(): string {
  const bytes = new Uint8Array(16);
  crypto.getRandomValues(bytes);
  return Array.from(bytes, (b) => b.toString(16).padStart(2, '0')).join('');
}

async function sha256Hex(input: string): Promise<string> {
  const data = new TextEncoder().encode(input);
  const digest = await crypto.subtle.digest('SHA-256', data);

```

```
return Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, '0')).join('');
}
```

Part C — Desktop: trust decision store

FILE: apps/desktop/src/renderer/src/services/trustDecisionStore.ts

```
import {
  evaluateTrust,
  statusFromPromptResponse,
  type DeviceFingerprint,
  type DeviceTrustStatus,
  type TrustEvaluation,
  type TrustPromptResponse,
} from '@remotedesk/shared';

/**
 * Orchestrates the T0FU flow on the desktop. Evaluates trust from the persisted
 * status + known fingerprint vs the freshly built one, exposes whether a prompt
 * is needed, and persists the operator's decision back to the API. Fail-closed:
 * the effectiveStatus is always untrusted (or blocked) until a prompt is resolved.
 */
export interface TrustPersistence {
  /** Persist the operator's trust decision + current fingerprint to the API. */
  persist: (deviceId: string, status: DeviceTrustStatus, fingerprint: DeviceFingerprint) => Promise<void>;
}

export class TrustDecisionStore {
  private evaluation: TrustEvaluation | null = null;
  private listeners = new Set<(e: TrustEvaluation) => void>();

  constructor(private readonly persistence: TrustPersistence) {}

  /** Evaluate trust for this connection. Returns the evaluation (also cached). */
  evaluate(input: {
    persistedStatus: DeviceTrustStatus | null;
    knownFingerprint: DeviceFingerprint | null;
    currentFingerprint: DeviceFingerprint;
  }): TrustEvaluation {
    this.evaluation = evaluateTrust(input);
    this.emit();
    return this.evaluation;
  }

  current(): TrustEvaluation | null {
    return this.evaluation;
  }

  /** Does the UI need to show a trust prompt right now? */
  needsPrompt(): boolean {
    return this.evaluation?.outcome === 'prompt';
  }

  /** Effective trust status to feed the policy snapshot (fail-closed default). */
  effectiveStatus(): DeviceTrustStatus {
```

```

    return this.evaluation?.effectiveStatus ?? 'untrusted';
  }

  /** Resolve a pending prompt: persist + update the cached evaluation. */
  async resolvePrompt(
    deviceId: string,
    response: TrustPromptResponse,
    currentFingerprint: DeviceFingerprint,
  ): Promise<DeviceTrustStatus> {
    const status = statusFromPromptResponse(response);
    try {
      await this.persistence.persist(deviceId, status, currentFingerprint);
    } catch {
      // Persistence failed: stay fail-closed (do not optimistically trust).
      return this.effectiveStatus();
    }
    this.evaluation = {
      outcome: status === 'trusted' ? 'trusted' : status === 'blocked' ? 'blocked' : 'untrusted',
      effectiveStatus: status,
      reason: 'unchanged',
    };
    this.emit();
    return status;
  }

  subscribe(fn: (e: TrustEvaluation) => void): () => void {
    this.listeners.add(fn);
    return () => this.listeners.delete(fn);
  }

  private emit(): void {
    if (this.evaluation) for (const fn of this.listeners) fn(this.evaluation);
  }
}

```

Part D — Desktop: policy client wiring

FILE: apps/desktop/src/renderer/src/services/devicePolicyClient.ts [MODIFY]

```

// ...existing Pack 15 imports...
import type { DeviceFingerprint } from '@remotedesk/shared';

// ---- ADD: send the fingerprint hash with the policy fetch so the API can
// return the known fingerprint + persisted status for TOFU comparison. ----
//
// async fetchWithFingerprint(deviceId: string, fingerprint: DeviceFingerprint): Promise<{
//   policy: DeviceAccessPolicySnapshot | null;
//   knownFingerprint: DeviceFingerprint | null;
// }> {
//   try {
//     const res = await fetch(`${this.baseUrl}${DEVICE_POLICY_ENDPOINT(deviceId)}`, {
//       headers: {
//         authorization: `Bearer ${this.authToken}`,
//         'x-device-fingerprint': fingerprint.hash, // hash only; never the signals/installId raw
//       },

```

```
//    });
//    if (!res.ok) throw new Error(`policy fetch status ${res.status}`);
//    const body = await res.json();
//    return { policy: body.policy ?? body, knownFingerprint: body.knownFingerprint ?? null };
//  } catch (err) {
//    console.warn('[policy] fetch failed, failing closed', { deviceId });
//    return { policy: null, knownFingerprint: null };
//  }
// }
//
// NOTE: only the fingerprint HASH crosses the wire here; the raw signals/installId
// stay on the device. If your API does not yet store/return knownFingerprint,
// treat it as null (=> first_use prompt), which is the safe default.
```

Part E — UI

FILE: apps/desktop/src/renderer/src/components/TrustPrompt.tsx

```
import React from 'react';
import type { TrustEvaluation, TrustPromptResponse } from '@remotedesk/shared';

/**
 * Trust-on-first-use prompt. Shown when the trust evaluation outcome is 'prompt'
 * (first connect or a material fingerprint change). Until resolved, the device is
 * untrusted, so Pack 15 already denies remote input. Screen viewing is unaffected.
 */
export function TrustPrompt({
  evaluation,
  onRespond,
}: {
  evaluation: TrustEvaluation;
  onRespond: (r: TrustPromptResponse) => void;
}) {
  if (evaluation.outcome !== 'prompt') return null;

  const headline =
    evaluation.reason === 'fingerprint_changed'
      ? 'This device looks different from last time'
      : 'New device connecting';
  const sub =
    evaluation.reason === 'fingerprint_changed'
      ? 'Its identity changed since you last trusted it. Remote input stays disabled until you decide.'
      : 'You have not trusted this device before. Remote input stays disabled until you decide.';

  return (
    <div className="trust-prompt" role="dialog" aria-modal="false">
      <h3 className="trust-prompt__title">{headline}</h3>
      <p className="trust-prompt__sub">{sub}</p>
      <div className="trust-prompt__actions">
        <button type="button" className="btn btn--primary" onClick={() => onRespond('trust')}>Trust this device</button>
        <button type="button" className="btn btn--ghost" onClick={() => onRespond('keep_untrusted')}>Keep untrusted (view
only)</button>
        <button type="button" className="btn btn--danger" onClick={() => onRespond('block')}>Block</button>
      </div>
    </div>
  );
}
```

```
);  
}
```

FILE: apps/desktop/src/renderer/src/components/FingerprintChangeBanner.tsx

```
import React from 'react';  
import { materialSignalChanges, type DeviceFingerprint } from '@remotedesk/shared';  
  
/**  
 * Compact banner noting which signals changed since the last trusted fingerprint.  
 * Non-sensitive (signal names only, never values like installId contents).  
 */  
export function FingerprintChangeBanner({  
  known,  
  current,  
}: {  
  known: DeviceFingerprint | null;  
  current: DeviceFingerprint | null;  
}) {  
  if (!known || !current) return null;  
  const changes = materialSignalChanges(known.signals, current.signals);  
  if (changes.length === 0) return null;  
  return (  
    <div className="fingerprint-change-banner" role="status">  
      Device identity changed: {changes.join(', ')}. Re-approval required before remote input.  
    </div>  
  );  
}
```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```
// ...existing imports + Pack 15→19 additions...  
import { TrustPrompt } from './TrustPrompt';  
import { FingerprintChangeBanner } from './FingerprintChangeBanner';  
import { buildFingerprint } from '../services/fingerprintBuilder';  
import type { DeviceFingerprint, TrustEvaluation, TrustPromptResponse } from '@remotedesk/shared';  
  
// ---- ADD: at session start, build fingerprint + evaluate trust BEFORE feeding  
// the policy's trustStatus into the host permission state ----  
//  
// const fp = await buildFingerprint(appVersion);  
// const { policy, knownFingerprint } = await devicePolicyClient.fetchWithFingerprint(deviceId, fp);  
// const evaluation = trustDecisionStore.evaluate({  
//   persistedStatus: policy?.trustStatus ?? null,  
//   knownFingerprint,  
//   currentFingerprint: fp,  
// });  
//  
// IMPORTANT: feed the EFFECTIVE trust status (not the raw persisted one) into the  
// policy snapshot, so a first-use/changed device is treated as untrusted:  
// hostPermissionState.setPolicy(policy ? { ...policy, trustStatus: evaluation.effectiveStatus } : null);  
  
// ---- ADD to render: trust prompt + fingerprint-change banner (host side) ----  
// {trustEval?.outcome === 'prompt' && (  
//   <TrustPrompt  
//     evaluation={trustEval}
```

```
//      onResponse={async (r: TrustPromptResponse) => {
//          const status = await trustDecisionStore.resolvePrompt(deviceId, r, fp);
//          hostPermissionState.setPolicy(policy ? { ...policy, trustStatus: status } : null);
//      }}
//  />
// })
// <FingerprintChangeBanner known={knownFingerprint} current={fp} />
//
// All Pack 15→19 UI (banners, prompts, panels, meters) stays as-is. Screen view
// is never gated by trust — only interactive features are.
```

Part F — Tests

FILE: tests/permissions/deviceFingerprint.test.ts

```
import { describe, it, expect } from 'vitest';
import {
    canonicalizeSignals,
    materialSignalChanges,
    isMaterialChange,
    type FingerprintSignals,
} from '@remotedesk/shared';

function signals(over: Partial<FingerprintSignals> = {}): FingerprintSignals {
    return { platformFamily: 'mac', appVersion: '1.4.0', screenBucket: '1920x1080', locale: 'en-US', installId: 'abc',
    ...over };
}

describe('deviceFingerprint', () => {
    it('canonicalizes signals stably', () => {
        expect(canonicalizeSignals(signals())).toBe('platform=mac&app=1.4.0&screen=1920x1080&locale=en-US&install=abc');
    });

    it('flags installId + platformFamily as material changes', () => {
        expect(materialSignalChanges(signals(), signals({ installId: 'xyz' }))).toEqual(['installId']);
        expect(materialSignalChanges(signals(), signals({ platformFamily: 'win' }))).toEqual(['platformFamily']);
    });

    it('does NOT treat appVersion/screen/locale drift as material', () => {
        expect(isMaterialChange(signals(), signals({ appVersion: '1.5.0' }))).toBe(false);
        expect(isMaterialChange(signals(), signals({ screenBucket: '2560x1440' }))).toBe(false);
        expect(isMaterialChange(signals(), signals({ locale: 'fr-FR' }))).toBe(false);
    });

    it('detects a material change', () => {
        expect(isMaterialChange(signals(), signals({ installId: 'new' }))).toBe(true);
    });
});
```

FILE: tests/permissions/trustDecision.test.ts

```
import { describe, it, expect } from 'vitest';
import {
    evaluateTrust,
    statusFromPromptResponse,
```

```

    type DeviceFingerprint,
  } from '@remotedesk/shared';

function fp(installId = 'abc', platformFamily = 'mac'): DeviceFingerprint {
  return {
    hash: `h-${installId}-${platformFamily}`,
    signals: { platformFamily, appVersion: '1.4.0', screenBucket: '1920x1080', locale: 'en-US', installId },
    computedAt: new Date().toISOString(),
  };
}

describe('evaluateTrust (TOFU)', () => {
  it('prompts + untrusted on first use (unknown device)', () => {
    const e = evaluateTrust({ persistedStatus: null, knownFingerprint: null, currentFingerprint: fp() });
    expect(e.outcome).toBe('prompt');
    expect(e.effectiveStatus).toBe('untrusted');
    expect(e.reason).toBe('first_use');
  });

  it('keeps trusted when known + fingerprint unchanged', () => {
    const e = evaluateTrust({ persistedStatus: 'trusted', knownFingerprint: fp(), currentFingerprint: fp() });
    expect(e.outcome).toBe('trusted');
    expect(e.effectiveStatus).toBe('trusted');
  });

  it('re-prompts + forces untrusted on a material fingerprint change', () => {
    const e = evaluateTrust({ persistedStatus: 'trusted', knownFingerprint: fp('abc'), currentFingerprint: fp('xyz') });
    expect(e.outcome).toBe('prompt');
    expect(e.effectiveStatus).toBe('untrusted');
    expect(e.reason).toBe('fingerprint_changed');
  });

  it('blocked is sticky and overrides everything', () => {
    const e = evaluateTrust({ persistedStatus: 'blocked', knownFingerprint: fp(), currentFingerprint: fp('xyz') });
    expect(e.outcome).toBe('blocked');
    expect(e.effectiveStatus).toBe('blocked');
  });

  it('keeps untrusted when known-but-untrusted + unchanged', () => {
    const e = evaluateTrust({ persistedStatus: 'untrusted', knownFingerprint: fp(), currentFingerprint: fp() });
    expect(e.outcome).toBe('untrusted');
    expect(e.effectiveStatus).toBe('untrusted');
  });

  it('maps prompt responses to statuses', () => {
    expect(statusFromPromptResponse('trust')).toBe('trusted');
    expect(statusFromPromptResponse('block')).toBe('blocked');
    expect(statusFromPromptResponse('keep_untrusted')).toBe('untrusted');
  });
});

```

FILE: tests/permissions/trustDecisionStore.test.ts

```

import { describe, it, expect, vi } from 'vitest';
import { TrustDecisionStore } from '../../apps/desktop/src/renderer/src/services/trustDecisionStore';
import type { DeviceFingerprint } from '@remotedesk/shared';

```

```

function fp(installId = 'abc'): DeviceFingerprint {
  return {
    hash: `h-${installId}`,
    signals: { platformFamily: 'mac', appVersion: '1.4.0', screenBucket: '1920x1080', locale: 'en-US', installId },
    computedAt: new Date().toISOString(),
  };
}

describe('TrustDecisionStore', () => {
  it('needs a prompt + reports untrusted on first use', () => {
    const store = new TrustDecisionStore({ persist: vi.fn().mockResolvedValue(undefined) });
    store.evaluate({ persistedStatus: null, knownFingerprint: null, currentFingerprint: fp() });
    expect(store.needsPrompt()).toBe(true);
    expect(store.effectiveStatus()).toBe('untrusted');
  });

  it('persists + flips to trusted when the operator trusts', async () => {
    const persist = vi.fn().mockResolvedValue(undefined);
    const store = new TrustDecisionStore({ persist });
    store.evaluate({ persistedStatus: null, knownFingerprint: null, currentFingerprint: fp() });
    const status = await store.resolvePrompt('dev-1', 'trust', fp());
    expect(status).toBe('trusted');
    expect(persist).toHaveBeenCalledWith('dev-1', 'trusted', expect.any(Object));
    expect(store.needsPrompt()).toBe(false);
    expect(store.effectiveStatus()).toBe('trusted');
  });

  it('stays fail-closed when persistence fails', async () => {
    const store = new TrustDecisionStore({ persist: vi.fn().mockRejectedValue(new Error('offline')) });
    store.evaluate({ persistedStatus: null, knownFingerprint: null, currentFingerprint: fp() });
    const status = await store.resolvePrompt('dev-1', 'trust', fp());
    expect(status).toBe('untrusted'); // not optimistically trusted
    expect(store.effectiveStatus()).toBe('untrusted');
  });

  it('block response persists blocked', async () => {
    const persist = vi.fn().mockResolvedValue(undefined);
    const store = new TrustDecisionStore({ persist });
    store.evaluate({ persistedStatus: null, knownFingerprint: null, currentFingerprint: fp() });
    const status = await store.resolvePrompt('dev-1', 'block', fp());
    expect(status).toBe('blocked');
    expect(store.effectiveStatus()).toBe('blocked');
  });
});

```

Part G — Reports

FILE: [generated-pack20-desktop-fingerprint-trust-merge-summary.md](#)

Pack 20 (Desktop) — Device Fingerprint + Trust-on-First-Use · Merge Summary

What this slice does

Continues Desktop Packs 15 → 19. Establishes + verifies device identity before trust is granted:

1. A non-sensitive device fingerprint (platform family, app version, coarse screen bucket, locale, a per-install random id) — NO IPs / serials / usernames.
2. Trust-on-first-use: first connect => prompt (untrusted until approved); a material fingerprint change on a known device => re-prompt + forced untrusted.
3. A trust prompt UI + fingerprint-change banner; the operator can trust, keep untrusted, or block. Decisions persist to the API.

The effective trust status feeds straight into Pack 15's policy snapshot, so an untrusted/changed device already has remote input denied. Screen viewing is never gated by trust.

Files created (9)

- packages/shared/src/permissions/deviceFingerprint.ts
- packages/shared/src/permissions/trustDecision.ts
- apps/desktop/src/renderer/src/services/fingerprintBuilder.ts
- apps/desktop/src/renderer/src/services/trustDecisionStore.ts
- apps/desktop/src/renderer/src/components/TrustPrompt.tsx
- apps/desktop/src/renderer/src/components/FingerprintChangeBanner.tsx
- tests/permissions/deviceFingerprint.test.ts
- tests/permissions/trustDecision.test.ts
- tests/permissions/trustDecisionStore.test.ts

Files modified (3)

- packages/shared/src/permissions/index.ts — export deviceFingerprint + trustDecision.
- apps/desktop/src/renderer/src/services/devicePolicyClient.ts — send fingerprint hash, receive knownFingerprint.
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — trust prompt + change banner + effective-status feed.

Files intentionally NOT touched

- main/index.ts, preload/index.ts — no native executor / no hardware serial access / no bridge.
- webrtc.ts, permissionState.ts — Pack 15→17 gates unchanged (permissionState just consumes trustStatus).
- apps/api/**, apps/web/** — trust persistence already exists; only the fingerprint field is new (optional).

Remaining manual wiring risks

1. API: store + return knownFingerprint alongside the device's trustStatus; accept a persist call with (status, fingerprint). If absent, knownFingerprint=null => safe first_use prompt.
2. At session start, build fingerprint -> evaluate -> feed EFFECTIVE status into the policy snapshot (not the raw persisted status).
3. Provide TrustPersistence.persist wired to your device-trust update endpoint.
4. Optional: a "forget this device" action calling resetInstallId().

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: first connect => trust prompt, input disabled until "Trust"; reinstall (installId change) on a trusted device => re-prompt + input disabled until re-approved; block => sticky.

FILE: [generated-pack20-desktop-fingerprint-trust-code-review.md](#)

Pack 20 (Desktop) — Code Review Notes

Strengths

- TOFU logic is pure + fail-closed: unknown or materially-changed device => untrusted until explicitly approved; blocked is sticky.
- Fingerprint is privacy-conscious: coarse, non-identifying signals only; only the HASH crosses the wire.
- Effective trust status flows into the EXISTING Pack 15 snapshot, so no new enforcement path — untrusted already denies input.
- Persistence failure stays fail-closed (never optimistically trusts).

- Pure logic + store fully unit-tested; no native/hardware access introduced.

Reviewer checklist

- [] API returns knownFingerprint + accepts (status, fingerprint) persistence; null known => first_use.
- [] Session start feeds evaluation.effectiveStatus into the policy snapshot, not the raw persisted status.
- [] Only the fingerprint hash is sent (header); raw signals/installId stay local.
- [] resetInstallId wired to any "forget device" UI if desired.
- [] No fingerprint signal values logged (names only in the change banner).

Safety

- No IPs, MAC/serials, usernames, or hardware ids in the fingerprint or logs.
- Untrusted/changed/unknown device cannot get remote input (Pack 15 denies it).
- Native OS execution remains disabled/no-op (untouched).

FILE: generated-pack20-desktop-fingerprint-trust-manifest.json

```
{
  "pack": "pack20-desktop-fingerprint-trust",
  "title": "First-Connect Device Fingerprint + Viewer Trust Prompt + Trust-on-First-Use",
  "actualFileCount": 12,
  "filesCreated": [
    "packages/shared/src/permissions/deviceFingerprint.ts",
    "packages/shared/src/permissions/trustDecision.ts",
    "apps/desktop/src/renderer/src/services/fingerprintBuilder.ts",
    "apps/desktop/src/renderer/src/services/trustDecisionStore.ts",
    "apps/desktop/src/renderer/src/components/TrustPrompt.tsx",
    "apps/desktop/src/renderer/src/components/FingerprintChangeBanner.tsx",
    "tests/permissions/deviceFingerprint.test.ts",
    "tests/permissions/trustDecision.test.ts",
    "tests/permissions/trustDecisionStore.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/devicePolicyClient.ts",
    "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/src/main/index.ts",
    "apps/desktop/src/preload/index.ts",
    "apps/desktop/src/renderer/src/services/webRTC.ts",
    "apps/desktop/src/renderer/src/services/permissionState.ts",
    "apps/api/**",
    "apps/web/**"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "react", "vitest"],
    "assumedFromProject": [
      "Desktop Pack 15 HostPermissionState + devicePolicyClient",
      "web crypto (crypto.subtle, getRandomValues) in the renderer",
      "localStorage in the renderer",
      "existing API device-trust update endpoint"
    ]
  },
  "knownLimitations": [
    "API must store/return a per-device fingerprint to detect changes; without it every connect is first_use (safe but
```

```
re-prompts).",
    "Fingerprint is intentionally coarse: it recognizes installs, it does not uniquely identify hardware.",
    "installId lives in localStorage; clearing storage looks like a new device (re-prompt).",
    "Native OS input execution remains disabled/no-op."
],
"remainingManualWiringRisks": [
    "API: persist + return knownFingerprint with trustStatus.",
    "Feed evaluation.effectiveStatus into the policy snapshot at session start.",
    "Wire TrustPersistence.persist to the device-trust endpoint."
],
"estimatedCompletionAfterCarefulMerge": "70-76%"
}
```

Desktop Pack 20 done: 12 files (9 created, 3 modified). This is the piece every earlier pack assumed existed: the moment a device's trust is first established. A privacy-conscious fingerprint (platform family, app version, coarse screen bucket, locale, a resettable per-install id, no IPs/serials/usernames) feeds a trust-on-first-use flow: first connect prompts and stays untrusted until approved, and a material fingerprint change on a known device forces a re-prompt and drops back to untrusted.

The clean bit: there's no new enforcement path. The trust evaluation just produces an `effectiveStatus` that flows into Pack 15's existing policy snapshot, so an unknown, changed, or blocked device already has remote input denied by machinery that's been in place for five packs. Persistence failure stays fail-closed (never optimistically trusts), blocked is sticky, and only the fingerprint hash ever crosses the wire.

That closes the trust lifecycle loop: Pack 20 establishes trust, 15 enforces it, 16 adds consent, 17 funnels input, 18 re-checks on reconnect, 19 keeps transfers/clipboard safe across it. Next natural slice = unattended-access pre-authorization (using the now-trusted device), or a session-summary/consent-receipt at disconnect. Bolo.